Assignment 1: Design Doc

Lexical & Semantic Retrieval on EB-NeRD and MIND

Yash More, 2024114004

Abstract

This report describes a retrieval pipeline built and measured on two news recommendation datasets, MIND (English) and EB-NeRD (Danish). Given a user's past clicks, the system retrieves articles they are likely to click next, using a lexical approach based on word overlap (BM25) and a semantic one based on pre-trained embeddings. Both are evaluated offline with the official ranking metrics and submitted to the two Codabench leaderboards. The main finding is that neither approach is universally better: BM25 wins on MIND and the semantic method wins on EB-NeRD, and the reason comes down to how good each dataset's provided embeddings are rather than anything inherent to lexical or semantic retrieval. Alongside accuracy, the engineering cost of each choice is measured rather than assumed.

1 Introduction

Background: The datasets used here record the moments a news app decides which articles to put in front of a user. Each record, called an *impression*, is one instance of a fixed list of candidate articles being shown to one user at one timestamp, together with which of those candidates the user clicked and which they ignored. Every impression also carries that user's click history up to that moment, the only real signal available about their interests.

Two Tasks: The task is then: given a user's click history as of the moment an impression fired, find articles they are likely to click and check that against what they actually clicked. This splits into two jobs. **The first is *retrieval*:** searching the whole catalog, which can exceed a hundred thousand articles, and pulling out a few hundred plausible ones, measured by recall@K in Sections 3 and 4. **The second is *ranking*:** ordering the roughly twenty candidates an impression showed, measured by AUC, MRR and nDCG in Section 5.

Datasets: MIND is English with about 65,000 articles in the development split; EB-NeRD is Danish with about 12,000 in the demo split, growing to over 125,000 in the real test set. One pipeline is parameterised per dataset, so when the two disagree the difference comes from the data, not from two implementations drifting apart.

2 Data Pipeline

Unified Schema: Both datasets are parsed into one common format so that everything downstream is written once. The raw formats differ: MIND stores history inside every impression row as a space-separated string of article identifiers, with no timestamps, while EB-NeRD keeps it in a separate per-user file joined on user identifier, with a timestamp for every historical click.

Temporal Split: The train, validation and test splits are drawn by time and never at random, which is the single most important correctness decision in the pipeline. For MIND the training file is cut by calendar day into train and validation, with the development file held out as the offline test set; EB-NeRD's provided windows are kept as they are. An automated test asserts on every split that the latest click timestamp in any history precedes the earliest impression timestamp, so this is checked mechanically rather than assumed.

Feature Store: The feature store holds one article table per dataset, carrying training click counts, and one user table per split. Those counts matter for cold start: with no usable history neither method has anything to build a query from, so the system falls back to training-set popularity rather than an all-zero score row. This happens for about 14% of MIND's evaluated impressions and never on EB-NeRD, where every evaluated user has prior history.

3 Lexical Retrieval

BM25 Scoring: The lexical method rests on a simple idea: if a user has been reading about a topic, other articles containing the same words are worth showing them. The titles of the last N articles they clicked are joined into one string and used as a query against the titles and abstracts of the whole catalog. Scoring uses BM25, which refines word counting so rare words count for more, repeated words saturate, and longer articles are penalised. The doc-side weight per article-word pair is

$$W[d,t] = idf(t) \cdot \frac{tf(t,d)\,(k_1+1)}{tf(t,d) + k_1\left(1 - b + b\,\frac{L_d}{\bar{L}}\right)},$$

where $tf(t,d)$ counts word t inside article d, $idf(t)$ measures how rare t is across the catalog, L_d is the article's length, $\bar{L}$ the average, and $k_1 = 1.5$ and $b = 0.75$ are the usual saturation and length-penalty constants.

Implementation: This was implemented directly rather than taken from a library. Every article-word weight is computed once into a sparse matrix W, one row per article and one column per vocabulary word, which for MIND is 65,238 by 60,849 and only 0.04% filled. Scoring a batch of queries is then a single sparse matrix multiplication, whose non-zero pattern is exactly the inverted index the assignment asks for. The alternative, `rank_bm25`, was rejected on measurement: scoring 196 real queries against MIND's catalog took 73.0 seconds with `rank_bm25`, which loops over articles one at a time in Python, against 0.09 seconds with the matrix version, about 782 times faster. On EB-NeRD the gap is about 497 times.

Tokenisation: Tokenisation is language-aware, because a tokeniser that strips non-ASCII characters would destroy Danish words containing æ, ø and å. Both languages use Unicode-aware splitting with their own stopword lists. Stemming was tested rather than assumed, and the answer depends on the language: it raises EB-NeRD's Pool B recall@200 from 0.0444 to 0.0604, because Danish builds long compound words a stemmer can break down, but drops MIND's from 0.1405 to 0.1372. It is therefore off for MIND, and could not be adopted for EB-NeRD in the time left for a Codabench re-evaluation, so both submissions are unstemmed.

Recall@K: Recall@K is reported under two candidate pools. Pool A searches the full catalog and measures raw retrieval power. Pool B restricts it to the union of articles that appeared in some impression's candidate list within that split, evaluated separately for validation and test, which is closer to a real re-ranker receiving a pre-filtered slate. Pool B recall is much higher at every cutoff, so the distinction is not merely academic; the absolute numbers are small in both cases, as expected given the size of the search space. Table 1 reports both methods.

4 Semantic Retrieval

Embeddings: The lexical method misses articles that share a topic but not vocabulary, since it matches literal words only. Semantic retrieval embeds each article as a vector positioned near others about similar things, then retrieves the vectors closest to one built from the user's history. Only the embeddings shipped with each dataset are used; none are trained here. MIND provides `entity_embedding.vec`, vectors for named entities keyed by Wikidata identifier, so an article vector is the average of the entities tagged in its title and abstract, covering 87.03% of the catalog. EB-NeRD provides a word2vec vector per article, at full coverage.

Search Index: Search uses FAISS's `IndexFlatIP`, an exact index that computes cosine similarity against every article vector, so it returns the true nearest neighbours.

Query Construction: A user's query vector is the average of the embeddings of their last N clicked articles. The validation sweep chose $N = 1$ for the semantic method on both datasets, against $N = 20$ for BM25 on MIND. Averaging many embeddings pulls the result toward the middle of everything a user has ever read, describing them in general but not what they want now, so a single recent click is the sharper signal. BM25 does not suffer this, because extra history words only add chances to match rather than blurring one point in space.

Recall Comparison: Table 1 gives a mixed picture. BM25 is better at every cutoff on MIND. On EB-NeRD the semantic method leads throughout Pool A and at B@50, but BM25 overtakes it from B@100 onward and is more than twice as good at B@200. Recall@K over the whole catalog and the ranking metrics in Section 5 are different questions, so a method leading on one need not lead on the other.

Approximate Search: Exact search is fast enough here, at roughly 1140 queries per second on MIND and 2580 on EB-NeRD at cutoff 200, so there is no reason to approximate at this scale. What approximate indexes would buy was still measured, in Table 2: both are more than ten times faster while returning over 90% of the same top-100 articles. That is the option when the catalog grows, since exact search costs time proportional to catalog size on every query.

5 Evaluation Harness

Accuracy Metrics: The harness answers the ranking question rather than the retrieval one: for each impression it takes only the candidates shown, scores and sorts them, and measures where the clicked candidate landed. AUC asks whether a randomly chosen clicked candidate outscores a randomly chosen unclicked one, so 0.5 means no better than shuffling. MRR rewards putting the first clicked candidate first. nDCG at cutoffs 5 and 10 rewards placing clicked candidates near the top, discounted by position. Every metric carries a bootstrap 95% confidence interval from 1000 seeded resamples, so small differences can be judged rather than over-read.

Beyond-Accuracy Metrics: Accuracy alone does not describe a recommender fully, so three further metrics are computed over each impression's top ten, in Table 3. Intra-list diversity (ILD) measures how different the recommended articles are from each other, from both categories and embeddings; novelty measures how obscure they are, using inverse training popularity; coverage measures what fraction of the catalog ever reaches anyone's top ten. Together they guard against recommending ten near-identical, already-popular articles from the same small set.

Slicing: Results are also broken down by slice, since an average can hide a method being good for one group and bad for another: cold-start against warm users, split at fewer than five prior clicks, and head against tail articles, split at the 90th percentile of training click count. That second split needed care, since more than 90% of articles have zero training clicks on both datasets, so the percentile lands on zero and a naive "at least zero" comparison would have labelled everything popular, leaving the tail empty. A strict greater-than comparison fixes this, guarded by a regression test; the lower half of Table 3 reports all four slices.

Leaderboard Results: Both submissions were generated from these same scoring functions and uploaded to their respective Codabench leaderboards, scoring 0.5851 AUC on MIND and 0.514 on EB-NeRD. Screenshots of both scored submissions are in the `docs/screenshots/` folder of the repository.

6 Observations

Dataset-Dependent Winner: The two datasets disagree about which method is better, consistently across every accuracy metric in Table 3: BM25 wins on MIND, the semantic method wins on EB-NeRD. This looks like evidence

	A@50	A@100	A@200	B@50	B@100	B@200
MIND (BM25)	0.80%	1.53%	2.63%	4.88%	7.19%	10.80%
MIND (semantic)	0.34%	0.51%	0.97%	2.11%	3.68%	6.41%
EB-NeRD (BM25)	1.01%	1.63%	2.35%	3.32%	9.83%	30.50%
EB-NeRD (semantic)	1.09%	2.14%	3.97%	4.26%	7.68%	14.40%

Table 1: Recall@K on the held-out test split for both retrieval methods, under both candidate pools (Pool A full catalog, Pool B shown-articles only).

Dataset	Index	Queries/s	Agreement
MIND	Exact	3,624	1.000
MIND	IVF (nprobe 8)	46,476	0.913
MIND	HNSW (ef 64)	31,940	0.931
EB-NeRD	Exact	2,150	1.000
EB-NeRD	IVF (nprobe 8)	43,153	0.925
EB-NeRD	HNSW (ef 64)	40,075	0.992

Table 2: Approximate nearest-neighbour indexes against exact search. Agreement is the fraction of the exact index's top-100 that the approximate index also returns.

about lexical versus semantic retrieval in general, but the likelier explanation is the embeddings each provides. EB-NeRD's word2vec vectors were trained on the same Danish news corpus and cover every article, so they genuinely describe content. MIND's describe named entities instead, cover 87% of the catalog, and represent an article by averaging whatever people and organisations it mentions, a much weaker signal competing against BM25 in English where exact word matching already works well.

Cold-Start Users: On MIND, BM25 improves from 0.520 AUC on cold-start users to 0.560 on warm ones, which is what should happen: more history means a longer query and more chances to match. The semantic method barely moves, 0.527 to 0.538, and is the better of the two on cold-start users specifically. This fits the averaging effect above: it gains little from extra history and can be hurt by it.

Popular Articles: The head and tail split is more surprising. On MIND, BM25 does clearly better on head articles, 0.586 AUC and 0.412 MRR on the popular tenth of the catalog, while the semantic method drops to 0.476 there, below random, yet manages 0.543 on the tail. On EB-NeRD the split falls between metrics rather than methods: on head articles the semantic method has better AUC, 0.551 against 0.495, but worse MRR, 0.295 against 0.433. It judges broad plausibility well, but is less reliable at picking the single right one.

Diversity vs. Accuracy: The beyond-accuracy columns in Table 3 run against the usual assumption that accuracy and diversity trade off: BM25 has both higher accuracy on MIND and higher diversity and coverage on both datasets. Embedding-based diversity on EB-NeRD is far below MIND's, 0.181 and 0.171 against 0.747 and 0.664, reflecting that its word2vec vectors place all articles from one newspaper much closer together. Coverage is low everywhere, 5% and 15%, so most of the catalog never reaches anyone's top ten.

7 Serving-Feature Ablation

Motivation: A model can look excellent offline by using information that would not exist when a real prediction has to be made, and this section measures how much of a strong-looking result comes from exactly that. It runs on EB-NeRD only, and not by choice: MIND's raw files carry no engagement or post-interaction statistics, so the comparison cannot be built there.

Configurations: Three configurations are compared in Table 4, each adding to the previous. The clean one uses only the retrieval score and is the only deployable one. The popularity one adds article statistics such as total views, read time and sentiment score, unavailable at serving time because they aggregate over an article's whole lifetime, including the future relative to any given impression. The post-interaction one adds a large bonus to whichever candidate was the true click, the most direct leak possible.

Results: Popularity lifts AUC by roughly 0.07 to 0.08 on both methods, a tempting improvement since popular articles genuinely do get clicked more. The post-interaction configuration scores a perfect 1.000 on every metric, by construction. The gap between the clean and popularity rows quantifies how much of a popularity-boosted score is leakage rather than retrieval quality.

A Caveat: Not every leaky column is exploitable. EB-NeRD also ships next read time and next scroll percentage, which describe what happened after an impression, but they are recorded per impression rather than per candidate, so every candidate gets the same value. Since AUC, MRR and nDCG depend only on relative ordering within an impression, a value identical across all candidates cannot change them at any weight.

8 Where It Breaks at Scale

Scale Jump: This section needed no extrapolation: the real Codabench test sets are an order of magnitude larger than the development data and broke the pipeline three times. EB-NeRD grows from 11,800 articles and 24,700 impressions to 125,500 and 13.5 million, 548 times more; MIND grows from 65,000 and 73,000 to 121,000 and 2.37 million.

Failure 1: Memory: Reading the full behaviors and history parquet files into pandas before any batching, including

Overall	AUC	MRR	nDCG@5	nDCG@10	ILD (cat.)	ILD (emb.)	Novelty	Coverage
MIND / BM25	**0.554**	**0.306**	**0.280**	**0.342**	**0.843**	**0.747**	16.99	**0.055**
MIND / semantic	0.536	0.274	0.252	0.314	0.827	0.664	**17.03**	0.049
EB-NeRD / BM25	0.501	0.267	0.283	0.383	**0.799**	**0.181**	14.35	**0.148**
EB-NeRD / semantic	**0.509**	**0.325**	**0.354**	**0.439**	0.781	0.171	**14.40**	0.139

By slice	Cold AUC	Warm AUC	Head AUC	Tail AUC	Head MRR	Tail MRR
MIND / BM25	0.520	**0.560**	**0.586**	**0.551**	**0.412**	**0.294**
MIND / semantic	**0.527**	0.538	0.476	0.543	0.345	0.265
EB-NeRD / BM25	–	0.501	0.495	0.501	**0.433**	0.265
EB-NeRD / semantic	–	**0.509**	**0.551**	**0.508**	0.295	**0.325**

Table 3: Ranking results on the offline test split, better value per dataset in bold. Upper: accuracy and beyond-accuracy metrics overall, where BM25 wins every accuracy metric on MIND and the semantic method wins every one on EB-NeRD. Lower: accuracy by slice, cold-start vs. warm users and head vs. tail articles. EB-NeRD has no cold-start users in its evaluated split, so its Cold column is marked –.

Method	Configuration	AUC	MRR	nDCG@10
BM25	clean	0.501	0.267	0.383
BM25	+ popularity	0.576	0.360	0.475
BM25	+ post-inter.	**1.000**	**1.000**	**1.000**
Semantic	clean	0.509	0.325	0.439
Semantic	+ popularity	0.581	0.363	0.479
Semantic	+ post-inter.	**1.000**	**1.000**	**1.000**

Table 4: Serving-feature ablation on EB-NeRD, 11,798 impressions. Only the clean rows are deployable; the perfect scores are produced by construction, which is the point.

many columns the pipeline never uses, reached 10.5 GB of resident memory on a 15 GB machine and was killed by the OS. Reading only the four needed columns and streaming impressions in batches brought peak memory to 1.7 GB.

Failure 2: Dictionary Construction: The scoring code built a dictionary from every article in the catalog to its score, once per impression. Harmless at MIND's development size, this was estimated at more than 70 hours at 125,500 articles and 13.5 million impressions, where each impression has only about twelve real candidates. The fix was to build it over each batch's own candidates, using a precomputed article-to-column mapping.

Failure 3: Matrix Size: Even after that fix, the full similarity matrix between each batch and the entire catalog was estimated at 10 hours, measured at 367 impressions per second. Because that matrix's columns are independent, restricting the multiplication to only the 250 to 300 candidate columns a batch touches is numerically identical rather than approximate; the largest difference measured was 1.8×10^{-7}, ordinary 32-bit rounding. The run dropped to 49 minutes, or 4,629 impressions per second.

Thread Scaling: Thread count was measured on the real matrix shape rather than assumed from numpy's printed build configuration. The dense multiplication keeps improving past two threads, from 180 to 95 to 52 milliseconds at one, two and four, plateauing there. BM25's sparse multiplication shows no thread sensitivity at all, flat at about 2,150 queries per second from one to eight threads, because scipy's sparse routines do not use the threaded dense path. The bottleneck was matrix size alone.

Applying the Fixes: Applying all three fixes preemptively in MIND's submission script produced a clean first run at 0.83 GB and 2,019 impressions per second. A fourth, unrelated problem: EB-NeRD's 1.6 GB test set downloaded at only 15 to 30 KB/s over a single connection, so splitting it across 24 parallel range requests raised throughput roughly thirtyfold.

Looking Further Ahead: EB-NeRD's full production data is around 600 million impressions, 44 times beyond what was tested, where even the fixed scoring loop would need roughly 36 hours: once the matrix multiplication is no longer dominant, the per-impression Python loop becomes the bottleneck and would need sharding or full vectorisation. Article embeddings stay manageable at ten times the catalog size, around 1.5 GB, but need streaming at a hundred times.

9 Limitations

Everything here runs on a single machine with no GPU. Embeddings are used as provided rather than trained, an explicit instruction that caps the semantic method at whatever each dataset shipped. The approximate-index comparison in Table 2 was measured at development scale only. Cold-start handling is limited to training-popularity fallback, with no exploration, and stemming helps EB-NeRD but is not in the scored submission. Both Codabench test sets are blind, so submissions could only be checked structurally before scoring.

Code

The full implementation, including the data pipeline, both retrieval methods, the evaluation harness and the Codabench submission scripts, is available at https://github.com/Yash-More1224/retrival-systems.